

Documentacion tecnica

Proyecto CV LaTeX Builder - Documentacion Tecnica

Resumen del proyecto

CV LaTeX Builder es una aplicacion web local para gestionar CVs, cartas de presentacion, postulaciones laborales y un chequeo ATS basico sin servicios externos. El MVP fue construido para ejecutarse con Docker Compose, persistir datos en SQLite y generar artefactos TEX, PDF y JSON de forma controlada.

Objetivo del MVP

- Centralizar la gestion documental de una busqueda laboral en una sola web local.
- Mantener una base portable y reproducible, sin depender de cloud ni autenticacion.
- Validar un flujo completo de CRUD, exportaciones, importaciones y analisis ATS basico.

Arquitectura general

Area	Implementacion
Backend	FastAPI con rutas modulares
Render web	Jinja2 con sidebar fija, dashboard operativo, HTML server-side y CSS con dark/light mode
Persistencia	SQLite en <code>/data/app.db</code>
Exportaciones	Servicios Python para TEX, PDF y JSON
PDF	pdflatex dentro del contenedor
Assets estaticos	<code>app/static</code> servido por FastAPI

Modulos implementados

CV Builder Core

- CRUD completo de CVs.
- Duplicado con validacion de titulo.
- Eliminacion logica con confirmacion.
- Exportaciones TEX, PDF y JSON.
- Importacion JSON con limite de tamano.

Cartas de presentacion

- CRUD basico de cartas.
- Asociacion opcional a un CV existente.
- Exportaciones TEX y PDF.

Postulaciones

- CRUD basico de postulaciones.
- Asociacion opcional a CV y carta.
- Estados: pendiente, enviado, entrevista, rechazado, oferta y pausado.

ATS Basic Check

- Checklist deterministico sobre CVs guardados.
- Score simple, advertencias y recomendaciones.
- Penalizacion de secciones criticas faltantes.

UI privada y dashboard operativo

- Sidebar fija para navegar entre Dashboard, CVs, Cartas, Postulaciones, ATS y Documentacion.
- Toggle dark/light persistido en `localStorage`.
- Dashboard enfocado en CVs y cartas como modulos principales.
- Listados compactos con preview visual CSS de documento y acciones secundarias.
- Eliminacion segura de CVs y cartas exigiendo coincidencia exacta del texto mostrado.

Docker Compose

- Servicio principal app con FastAPI.
- Publicacion local por defecto en `127.0.0.1:8000`.
- Healthcheck HTTP sobre `/health`.
- Reinicio `unless-stopped`.

SQLite y persistencia

- La base se inicializa en `/data/app.db`.
- El host monta `./data:/data`.
- Las tablas activas son `cvs`, `cover_letters` y `applications`.
- Se usa `deleted_at` para eliminacion logica en entidades funcionales.

Exports TEX, PDF y JSON

- CVs: TEX, PDF y JSON.
- Cartas: TEX y PDF.
- Los exports persistidos quedan en `/data/exports`.
- Los nombres de archivo se sanitizan y se acotan a 180 caracteres.
- La compilacion PDF ocurre en un temporal controlado.

Cartas de presentacion

El modulo reutiliza la misma infraestructura LaTeX/PDF del modulo CVs, con su propia plantilla `classic_letter.tex` y validaciones especificas para empresa, puesto, saludo, cuerpo y firma.

Postulaciones

El tracker de postulaciones mantiene una asociacion opcional a CV y cover letter, sin integraciones externas ni calendario. Es un modulo local de seguimiento operativo.

ATS Basic Check

El servicio ATS analiza presencia de email, telefono, resumen, experiencia, educacion y skills, ademas de una longitud aproximada del CV. Ningun CV con secciones core faltantes puede quedar en estado Bueno.

Git Flow usado

1. `main` como rama estable.
2. `development` como rama de integracion.
3. `feature/*` por etapa o mini-etapa.
4. Merge fast-forward solo despues de validacion explicita.

PRs y Codex Review

- Cada etapa integrada paso por rama feature dedicada.
- Se abrieron PRs hacia `development` para revision manual.
- Codex Review se uso como control de calidad antes de mergear.

Validaciones realizadas

- `python -m compileall app tests`
- `docker compose build`
- `docker compose up -d`

- `docker compose ps`
- `docker compose exec app python -m pytest`
- Validaciones HTTP y visuales sobre dashboard, modulos y exports
- Verificacion de extraccion PDF con `pdftotext`

Base estable y rama visual actual

- Base estable publicada: `tag v0.8.0`
- Commit base estable: `eab9556 fix(docs): render documentation as html with pdf downloads`
- Rama visual actual: `feature/ui-private-dashboard`
- Objetivo de la rama: rediseñar la experiencia privada sin tocar base de datos, IA ni autenticacion.

Riesgos conocidos

- No hay login ni permisos.
- No hay CSRF ni hardening para despliegue remoto.
- SQLite no aplica claves foraneas duras entre modulos.
- La imagen Docker incluye dependencias de LaTeX y test, por lo que es pesada.

Backlog posterior

- PR de la rama visual hacia `development` y `@codex review` manual.
- Documentacion PDF futura adicional si hiciera falta.
- Login/auth si el alcance deja de ser local.
- Migraciones o mayor integridad relacional si la persistencia crece.
- Mejoras de export ATS y coverage end-to-end.

Historial y rollback

- Referencia operativa: `docs/development/PROJECT_HISTORY_ROLLBACK.md`
- El archivo documenta el tag estable `v0.8.0`, el commit base `eab9556` y comandos de inspeccion o rollback temporal con advertencia de no ejecutarlos sin validacion previa.